

آموزش کار با پین در میکروپایتون - از پایه تا پیشرفته 🌱

📅 زمان برگزاری: آذر ماه ۱۴۰۴

📖 نام دوره: میکروپایتون مقدماتی

🎓 کارشناسی ارشد الکترونیک دیجیتال

👨‍🏫 مدرس: مهندس یاسین سموعی

سلام دوست من 🌱

در این آموزش قدم به قدم از مفاهیم پایه شروع می‌کنیم و تا کار با پین‌ها در میکروپایتون پیش می‌رویم. اگر اولین بار است با این مفاهیم آشنا می‌شوید، نگران نباشید! همه چیز را از صفر توضیح می‌دهیم.

🎯 بخش اول: درک مفهوم کلاس

مرحله 1:

♦ کلاس چیست؟ (ایده اصلی)

کلاس یعنی قالب، نقشه یا تعریف یک چیز. ما از روی کلاس، چیزهای واقعی می‌سازیم.

📖 مثال ملموس: الگوی لباس

- کلاس = الگوی لباس (فقط نقشه و طرح است)
- شیء (Object) = لباسی که دوختیم (چیز واقعی و قابل استفاده)

الگوی لباس فقط مشخص می‌کند:

- کجا آستین باشد
- چه سائزی باشد
- چه دکمه‌ای داشته باشد

اما لباس واقعی را می‌توانید:

- بپوشید

- دکمه‌اش را ببندید
- به رنگ‌های مختلف داشته باشید

✦ کلاس = تعریف و نقشه

✦ شیء = نمونه واقعی ساخته شده

مرحله 2:

♦ کلاس در پایتون چیست؟

در پایتون، کلاس یک قالب است که مشخص می‌کند یک شیء:

- چه ویژگی‌هایی دارد (مثل رنگ، مدل، شماره)
- چه کارهایی می‌تواند انجام دهد (مثل روشن شدن، خاموش شدن)

💡 مثال: کلاس لامپ

فرض کنید یک کلاس به نام **Lamp** داریم:

```
class Lamp:
    def __init__(self, color):
        self.color = color
        self.status = "off"

    def turn_on(self):
        self.status = "on"
        print("The lamp is turned on")

    def turn_off(self):
        self.status = "off"
        print("The lamp is turned off")
```

توجه: این فقط تعریف است! تا وقتی نمونه‌ای از آن نسازیم، کاری انجام نمی‌دهد.

مرحله 3:

♦ ساختن شیء از روی کلاس

وقتی از روی یک کلاس، یک نمونه واقعی می‌سازیم، به آن **ساختن شیء (Object)** می‌گوییم.

```
# ساخت دو لامپ از روی کلاس
lamp1 = Lamp("red") # لامپ قرمز
lamp2 = Lamp("blue") # لامپ آبی

# حالا می‌توانیم با آنها کار کنیم
lamp1.turn_on() # "خروجی: "لامپ روشن شد"
lamp2.turn_off() # "خروجی: "لامپ خاموش شد"
```

⚠ **نکته مهم:** در میکروپایتون هم دقیقاً همین منطق وجود دارد. ما کلاس‌هایی داریم که تعریف شده‌اند و باید از آنها شیء بسازیم تا بتوانیم با سخت‌افزار کار کنیم.

🎯 بخش دوم: پارامتر دادن به کلاس

مرحله 4:

♦ پارامتر چیست؟

بعضی کلاس‌ها موقع ساخته شدن، **اطلاعات اولیه** نیاز دارند. به این اطلاعات **پارامتر** می‌گوییم.

📄 مثال: ساختن خانه

وقتی می‌خواهید خانه بسازید، باید اطلاعاتی بدهید:

- خانه چند اتاق دارد؟
- چه رنگی است؟
- در کدام محله است؟

این اطلاعات همان **پارامترها** هستند که هنگام ساخت شیء به کلاس داده می‌شوند.

مرحله 5:

♦ پارامتر در کد پایتون

در کد زیر، `brand`، `model`، `year` سه پارامتر هستند:

```
class Car:
    def __init__(self, brand, model, year):
        self.brand = brand    # پارامتر اول
        self.model = model    # پارامتر دوم
        self.year = year      # پارامتر سوم

# ساختن دو ماشین با پارامترهای مختلف
car1 = Car("Toyota", "Corolla", 2018)
car2 = Car("BMW", "X3", 2021)
```

♦ **نکته طلایی:** کلاس **Pin** در میکروپایتون هم دقیقاً همین‌طور کار می‌کند! وقتی می‌خواهیم یک پین بسازیم، باید پارامترهای لازم را به آن بدهیم.

🎯 بخش سوم: Import کردن چیست؟

مرحله 6:

♦ مفهوم Import

import یعنی آوردن و استفاده از یک **جعبه ابزار آماده**.

🔧 مثال واقعی: فروشگاه قطعات الکترونیک

- شما در خانه همه قطعات الکترونیکی را ندارید
- می‌روید فروشگاه و یک سنسور دما می‌خرید
- حالا می‌توانید از آن در پروژه‌تان استفاده کنید

مرحله 7:

♦ تفاوت پکیج، کتابخانه و ماژول

📦 پکیج (Package)

یک مجموعه کامل از چندین کتابخانه مرتبط. مثل یک **فروشگاه کامل الکترونیک** که بخش‌های مختلف دارد: بخش سنسورها، بخش نمایشگرها، بخش ماژول‌های ارتباطی و غیره.

مثال: یک پکیج کامل IoT که شامل کتابخانه‌های WiFi، MQTT، سنسورها و نمایشگرها باشد.

📚 کتابخانه (Library)

مجموعه‌ای از توابع و کلاس‌های مرتبط با یک موضوع خاص. مثل یک **بخش خاص در فروشگاه الکترونیک** - مثلاً بخش سنسورها که همه انواع سنسور را دارد.

مثال: کتابخانه machine در میکروپایتون که شامل همه ابزارهای کار با سخت‌افزار است.

✂️ ماژول (Module)

یک فایل پایتون که شامل کدهای مرتبط است. مثل یک **قطعه یا محصول خاص** در فروشگاه - مثلاً یک سنسور دمای خاص با مدل مشخص.

مثال: ماژول Pin که فقط برای کار با پین‌های ورودی/خروجی است.

🖥️ مثال واقعی در میکروپایتون

```
# import کردن کل کتابخانه machine
# (مثل خرید همه قطعات بخش سخت‌افزار)
import machine

# import کردن فقط کلاس Pin
```

(مثل خرید فقط یک سنسور خاص)

```
from machine import Pin
```

تشبیه کامل:

- پکیج = فروشگاه کامل الکترونیک (چندین بخش دارد)
- کتابخانه = یک بخش خاص فروشگاه (مثل بخش سنسورها)
- ماژول = یک قطعه خاص (مثل سنسور DHT11)

بخش چهارم: ماژول Machine و کلاس Pin

مرحله 8:

ماژول Machine چیست؟

در میکروپایتون، یک جعبه ابزار مخصوص سخت‌افزار داریم به نام `machine`.

محتویات جعبه ابزار `machine`:

- **Pin** - برای کار با پایه‌های برد
- **PWM** - برای کنترل سرعت و نور
- **ADC** - برای خواندن مقادیر آنالوگ
- **Timer** - برای زمان‌بندی
- **I2C و SPI** - برای ارتباط با سنسورها

یعنی هر چیزی که به پایه‌های برد مربوط باشد، در این ماژول پیدا می‌شود.

مرحله 9:

کلاس Pin چیست؟ (بخش بسیار مهم)

Pin یک کلاس است که داخل ماژول `machine` تعریف شده است.

📌 یعنی چه؟

- یعنی تعریف شده است (نقشه کشیده شده)
- یعنی هنوز پین واقعی نداریم
- فقط می‌گویید پین چه کارهایی می‌تواند بکند

پین واقعی زمانی ساخته می‌شود که ما از این کلاس استفاده کنیم!

مرحله 10:

♦ ساختن یک پین واقعی

حالا همه مفاهیم را به هم وصل می‌کنیم:

```
# machine از جعبه ابزار Pin مرحله ۱: آوردن کلاس
from machine import Pin

# Pin مرحله ۲: ساختن یک پین واقعی (شیء) از روی کلاس
led = Pin(2, Pin.OUT)
```

✂ تحلیل این دو خط کد:

خط اول: "از جعبه ابزار machine، کلاس Pin را بیاور"

خط دوم: "از روی کلاس Pin، یک پین واقعی به نام led بساز"

🎯 بخش پنجم: پارامترهای کلاس Pin

مرحله 11:

♦ پارامتر اول: شماره پایه (Pin Number)

شماره فیزیکی پایه روی برد را مشخص می‌کند.

```
# پایه شماره ۲ برد ESP32
led = Pin(2, Pin.OUT)

# پایه شماره ۴ برد ESP32
sensor = Pin(4, Pin.IN)
```

⚠ **توجه:** شماره پایه‌ها در بردهای مختلف متفاوت است. حتما دیتاشیت برد خود را بررسی کنید.

مرحله 12:

♦ پارامتر دوم: حالت پایه (Mode)

مشخص می‌کند پایه ورودی است یا خروجی.

● حالت خروجی (OUT)

وقتی می‌خواهیم به بیرون فرمان بدهیم:

- روشن کردن LED
- کنترل رله
- بوق زدن با بیزر

```
led = Pin(2, Pin.OUT)
```

● حالت ورودی (IN)

وقتی می‌خواهیم از بیرون اطلاعات بگیریم:

- خواندن دکمه
- دریافت داده از سنسور PIR
- خواندن سنسور فاصله

```
button = Pin(4, Pin.IN)
```

♦ پارامترهای اختیاری دیگر

Pull-up/Pull-down resistors 🕒

```
# داخلی Pull-up فعال کردن مقاومت
button = Pin(4, Pin.IN, Pin.PULL_UP)

# داخلی Pull-down فعال کردن مقاومت
button = Pin(4, Pin.IN, Pin.PULL_DOWN)
```

کاربرد: برای پایه‌های ورودی که به دکمه یا سوئیچ متصل هستند.

🎯 بخش ششم: کار با پین‌های ساخته شده

♦ روشن و خاموش کردن (برای خروجی)

وقتی پین در حالت خروجی است، می‌توانیم آن را روشن یا خاموش کنیم:

```
from machine import Pin
import time

led = Pin(2, Pin.OUT)

# روشن کردن LED
led.on()
time.sleep(1) # یک ثانیه صبر کن

# خاموش کردن LED
led.off()
time.sleep(1)

# چشمک زن ساده
for i in range(10):
    led.on()
    time.sleep(0.5)
```

```
led.off()

time.sleep(0.5)
```

💡 **نکته:** `()on` و `()off` **متد** هستند. متد یعنی کاری که یک شیء می‌تواند انجام دهد.

مرحله 15:

♦ خواندن مقدار (برای ورودی)

وقتی پین در حالت ورودی است، می‌توانیم مقدار آن را بخوانیم:

```
from machine import Pin
import time

button = Pin(4, Pin.IN, Pin.PULL_UP)

while True:
    value = button.value() # خواندن مقدار پایه

    if value == 0: # PULL_UP به دلیل) اگر دکمه فشرده شد
        print("دکمه فشرده شد")

    time.sleep(0.1) # میلی‌ثانیه صبر کن 100
```

📁 مقدار `value()` چیست؟

- **0** = ولتاژ پایین (Low) ≈ 0 ولت
- **1** = ولتاژ بالا (High) ≈ 3.3 ولت

مرحله 16:

♦ نوشتن مقدار (برای خروجی)

می‌توانیم به جای `()on` و `()off`، از `value()` هم استفاده کنیم:

```
from machine import Pin
```

```
led = Pin(2, Pin.OUT)

# روشن کردن
led.value(1) # معادل led.on()

# خاموش کردن
led.value(0) # معادل led.off()

# حالت سومی وجود ندارد
# فقط 0 و 1 معنی دارند
```

🎯 **نکته کلیدی:** تابع `value()` دو کاره است:

- اگر پارامتر بگیرد: **می‌نویسد** (`led.value(1)`)
- اگر پارامتر نگیرد: **می‌خواند** (`btn_value = button.value()`)

🎯 بخش هفتم: جمع‌بندی و نکات نهایی

♦ خلاصه‌ی کل میکروپایتون برای کار با سخت‌افزار

در میکروپایتون، اساس کار با سخت‌افزار بر دو چیز استوار است:

۱. حالت‌های پایه

- **Pin.OUT** → وقتی می‌خواهید فرمان بدهید
- **Pin.IN** → وقتی می‌خواهید اطلاعات بگیرید

۲. توابع اصلی

- **on()** → روشن کردن خروجی
- **off()** → خاموش کردن خروجی
- **value()** → خواندن یا نوشتن مقدار

♦ پروژه‌ی عملی کامل

بیاید همه چیز را در یک پروژه عملی ببینیم:

```
# با دکمه LED پروژه: کنترل

from machine import Pin
import time

# تعریف پین‌ها
led = Pin(2, Pin.OUT)          # LED خروجی برای
button = Pin(4, Pin.IN, Pin.PULL_UP) # ورودی برای دکمه

print("شروع شد LED پروژه کنترل")
print("روشن شود LED دکمه را فشار دهید تا")

while True:
    # خواندن وضعیت دکمه
    button_state = button.value()

    if button_state == 0: # اگر دکمه فشرده شد
        led.on()          # LED را روشن کن
        print("✅ LED روشن")
    else:
        led.off()         # LED را خاموش کن
        print("❌ LED خاموش")

    time.sleep(0.1) # زیاد CPU جلوگیری از مصرف
```

💡 **چالش:** می‌توانید این کد را تغییر دهید تا:

- LED با هر بار فشار دکمه، حالتش تغییر کند (Toggle)
- اگر دکمه بیشتر از 2 ثانیه فشار داده شد، LED چشمک بزند
- از دو LED و دو دکمه استفاده کنید

-  **کلاس = قالب و نقشه** (مثل الگوی لباس)
-  **Pin** یک کلاس است داخل ماژول machine
-  **import** = آوردن ابزار آماده از جعبه ابزار
-  **machine** = جعبه ابزار سخت افزار میکروپایتون
-  **Pin.IN** = حالت ورودی (برای خواندن اطلاعات)
-  **Pin.OUT** = حالت خروجی (برای فرمان دادن)
-  **on()/off()** = فرمان روشن/خاموش
-  **value()** = خواندن یا نوشتن مقدار (0 یا 1)

حالا شما می دانید کلاس چیست و چگونه با پین ها در میکروپایتون کار کنید! 🎉